

```

const { Telegraf, Markup } = require("telegraf");
const fs = require("fs");
const path = require("path");
require("dotenv").config();
const OpenAI = require("openai");
const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

async function checkPaymentReceipt(imageUrl) {
  const response = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [
      {
        role: "system",
        content: `
You are a payment receipt verification AI.
Analyze payment receipt images from GCash, Maya, MariBank,
banks, and other e-wallets.
Do not require one fixed receipt format.
Do not reject a receipt only because a recipient name is masked,
abbreviated, contains dots, spaces, or has a different display format.
Determine whether the image appears to show a completed payment transaction.
Extract:
- payment method
- recipient
- recipient number if visible
- amount
- transaction date and time
- reference number / transaction ID
- transaction status
IMPORTANT:
- Never invent information that cannot be read.
- Use an empty string for unreadable/missing fields.
- "confidence" must be a number from 0 to 1.
- "is_receipt" should be true only when the image appears to be a payment receipt.
- For datetime, return ISO 8601 including timezone when you can determine it.
- If you are uncertain, lower the confidence instead of inventing data.
Return JSON only:
{
  "is_receipt": true,
  "payment_method": "",
  "recipient": "",
  "recipient_number": "",
  "amount": "",
  "datetime": "",
  "reference_number": "",
  "status": "",
  "confidence": 0
},
{
  role: "user",
  content: [
    {
      type: "text",
      text: "Analyze this payment receipt.",
    },
    {
      type: "image_url",
      image_url: {
        url: imageUrl,

```

```

    },
    },
  ],
},
],
});
return response.choices[0].message.content;
}

const BOT_TOKEN = process.env.BOT_TOKEN;
const ADMIN_ID = Number(process.env.ADMIN_ID || 0);
const CHANNEL_URL = process.env.CHANNEL_URL || "https://t.me/YOUR_CHANNEL";
const CHANNEL_ID = process.env.CHANNEL_ID || "";
const ADMIN_USERNAME = process.env.ADMIN_USERNAME || "YOUR_ADMIN_USERNAME";
const GCASH_NAME = process.env.GCASH_NAME || "GCash Account";
const GCASH_NUMBER = process.env.GCASH_NUMBER || "09XXXXXXXXX";
if (!BOT_TOKEN) throw new Error("Missing BOT_TOKEN in .env");
if (!ADMIN_ID) {
  console.warn("■■■■ ADMIN_ID is not set. Admin commands will not work.");
}

const bot = new Telegraf(BOT_TOKEN);
const DB_FILE = path.join(__dirname, "db.json");
const WELCOME_IMAGE = path.join(__dirname, "hyuna-welcome.png");
const PRODUCTS = {
  gemini: {
    id: "gemini",
    emoji: "■■■",
    name: "Gemini Pro / Flow",
    price: 100,
    details: ["1K Credits", "18 Months"],
    note: "■■■■ No warranty after claim",
    deliveryType: "link",
  },
  capcut: {
    id: "capcut",
    emoji: "■■■",
    name: "CapCut Pro",
    price: 150,
    details: ["1 Month"],
    note: "",
    deliveryType: "email_password",
  },
  chatgpt: {
    id: "chatgpt",
    emoji: "■■■",
    name: "ChatGPT Shared",
    price: 450,
    details: ["Shared by 4 persons", "1 device only", "Stable account"],
    note: "■■■ Full warranty ■■■ Manual account delivery up to 12 hours",
    deliveryType: "manual",
  },
  canva: {
    id: "canva",
    emoji: "■■■",
    name: "Canva Pro",
    price: 30,
    details: ["1 Month+", "Via invite"],
    note: "■■■ Send Gmail after payment ■■■ Manual delivery",
    deliveryType: "manual_invite",
  },
};
const pendingOrders = {};

```

```

const adminStockFlow = {};
const adminDelStockFlow = {};
const pendingReceiptOrders = {};
function loadDB() {
  if (!fs.existsSync(DB_FILE)) {
    const fresh = {
      orders: [],
      stock: [],
      users: [],
    };
    fs.writeFileSync(DB_FILE, JSON.stringify(fresh, null, 2));
    return fresh;
  }
  const raw = JSON.parse(fs.readFileSync(DB_FILE, "utf8"));
  if (raw.stock && !Array.isArray(raw.stock)) {
    const unified = [];
    for (const [pid, items] of Object.entries(raw.stock)) {
      if (!Array.isArray(items)) continue;
      for (const it of items) {
        if (!it || typeof it !== "object") continue;
        if (it.email && it.password) {
          unified.push({
            productId: pid,
            type: "email_password",
            email: it.email,
            password: it.password,
            addedAt: it.addedAt || new Date().toISOString(),
          });
        } else if (it.link) {
          unified.push({
            productId: pid,
            type: "link",
            link: it.link,
            addedAt: it.addedAt || new Date().toISOString(),
          });
        }
      }
    }
    raw.stock = unified;
    fs.writeFileSync(DB_FILE, JSON.stringify(raw, null, 2));
  }
  raw.stock = raw.stock || [];
  raw.orders = raw.orders || [];
  raw.users = raw.users || [];
  return raw;
}
function saveDB(db) {
  fs.writeFileSync(DB_FILE, JSON.stringify(db, null, 2));
}
async function notifyAllUsers(message) {
  const db = loadDB();
  for (const user of db.users) {
    await bot.telegram.sendMessage(user.id, message).catch(() => {});
  }
  if (CHANNEL_ID) {
    await bot.telegram.sendMessage(CHANNEL_ID, message).catch(() => {});
  }
}
function orderId() {
  const part = Math.random().toString(36).slice(2, 6).toUpperCase();

```

```

return `HYU-${part}`;
}
function isAdmin(ctx) {
const userId = Number(ctx.from?.id);
const isAllowed = userId === ADMIN_ID;
if (process.env.DEBUG_ADMIN) {
console.log(
`[ADMIN CHECK] User: ${userId}, ADMIN_ID: ${ADMIN_ID}, Allowed: ${isAllowed}`
);
}
return isAllowed;
}
function normalizeReference(value) {
return String(value || "").
replace(/\s+/g, "")
.trim();
}
function normalizePhone(value) {
return String(value || "").replace(/\D/g, "");
}
function parseReceiptAmount(value) {
const cleaned = String(value ?? "")
.replace(/,/g, "")
.replace(/^[^0-9.-]/g, "");
if (!cleaned) return NaN;
return Number(cleaned);
}
function menuKeyboard() {
return Markup.inlineKeyboard([
[
Markup.button.callback("🛒 Products", "products"),
Markup.button.callback("📦 My Orders", "my_orders"),
],
[
Markup.button.callback("💰 Payment Guide", "payment_guide"),
Markup.button.url("📺 Channel", CHANNEL_URL),
],
[Markup.button.url("👤 Contact Admin", `https://t.me/${ADMIN_USERNAME}`)],
]);
}
async function sendHome(ctx) {
const caption =
"🛒 *Welcome to Hyuna Store!* 🛒\n" +
"Your cute & trusted digital shop 🛒\n\n" +
"Choose an option below:";
if (fs.existsSync(WELCOME_IMAGE)) {
await ctx.replyWithPhoto(
{ source: WELCOME_IMAGE },
{
caption,
parse_mode: "Markdown",
...menuKeyboard(),
}
);
} else {
await ctx.reply(caption, {
parse_mode: "Markdown",
...menuKeyboard(),
});
}
}

```

```

bot.start(sendHome);
bot.command("menu", sendHome);
bot.action("products", async (ctx) => {
  await ctx.answerCbQuery();
  let text =
    "■■■■ *HYUNA STORE ■■ AVAILABLE PRODUCTS* ■■■■n" +
    "Choose your fave below ■■\n\n";
  for (const p of Object.values(PRODUCTS)) {
    text += `${p.emoji} *${p.name} ■■ ■■${p.price}*` + '\n';
    for (const d of p.details) {
      text += `■■ ${d}\n`;
    }
    if (p.note) {
      text += `${p.note}\n`;
    }
    text += "\n";
  }
  text += "■■ _Please read the product details before ordering._";
  await ctx.reply(text, {
    parse_mode: "Markdown",
    ..Markup.inlineKeyboard([
      [Markup.button.callback("■■ Buy Gemini", "buy:gemin"),],
      [Markup.button.callback("■■ Buy CapCut", "buy:capcut"),],
      [Markup.button.callback("■■ Buy ChatGPT", "buy:chatgpt"),],
      [Markup.button.callback("■■ Buy Canva", "buy:canva"),],
      [Markup.button.callback("■■■ Back to Menu", "home"),],
    ]),
  });
});
bot.action('home', async (ctx) => {
  await ctx.answerCbQuery()
  await sendHome(ctx)
})
bot.action(/^buy:(.+)$/, async (ctx) => {
  await ctx.answerCbQuery()
  const productId = ctx.match[1]
  const product = PRODUCTS[productId]
  if (!product) {
    return ctx.reply('Product not found.')
  }
  pendingOrders[ctx.from.id] = {
    productId,
    createdAt: new Date().toISOString()
  }
  await ctx.reply(
    '■■■ Ilan ang order? (1-50)\nHalimbawa: 5',
    {
      reply_markup: {
        force_reply: false
      }
    }
  )
})
bot.action(/^cancel:(HYU-.+)$/, async (ctx) => {
  await ctx.answerCbQuery()
  const id = ctx.match[1]
  const db = loadDB()
  const order = db.orders.find(
    o => o.id === id && o.userId === ctx.from.id
  )
})

```

```

if (!order) {
  return ctx.reply('Order not found.')
}
if (order.status !== 'waiting_payment') {
  return ctx.reply(
    'This order can no longer be cancelled.'
  )
}
order.status = 'cancelled'
saveDB(db)
delete pendingReceiptOrders[ctx.from.id]
await ctx.reply(
  `■■ Order \`${id}\` cancelled.`,
  { parse_mode: 'Markdown' }
)
})
bot.action('payment_guide', async (ctx) => {
  await ctx.answerCbQuery()
  const db = loadDB()
  const order = db.orders
    .filter(
      o =>
        o.userId === ctx.from.id &&
        o.status === 'waiting_payment'
    )
    .slice(-1)[0]
  let caption =
    '■■ *PAYMENT GUIDE* ■■\n\n' +
    `■■ GCash Name: ${GCASH_NAME}\n` +
    `■■ GCash Number: ${GCASH_NUMBER}\n\n` +
    'Scan the QR code to pay.\n\n' +
    '■■ Keep your Order ID for reference.\n\n'
  if (order) {
    caption =
      `■■ *PAYMENT SUMMARY*\n\n` +
      `Product: *${order.productName}*\n` +
      `Quantity: *${order.quantity}*\n` +
      `Price: *■■${order.pricePerItem} each*\n` +
      `Total: *■■${order.totalPrice}*\n\n` +
      caption
  }
  await ctx.replyWithPhoto(
    { source: './GCash-MyQR-08092026123724.PNG.jpg' },
    {
      caption,
      parse_mode: 'Markdown',
      ...(order
        ? Markup.inlineKeyboard([
            [
              Markup.button.callback(
                '■■ Send Receipt',
                `send_receipt:${order.id}`
              )
            ]
          ])
        : {})
    }
  )
  bot.action(/^send_receipt:(HYU-.+)$/, async (ctx) => {
    await ctx.answerCbQuery()
  })
})

```

```

const id = ctx.match[1]
const db = loadDB()
const order = db.orders.find(
  o => o.id === id && o.userId === ctx.from.id
)
if (!order) {
  return ctx.reply('Order not found.')
}
if (order.status !== 'waiting_payment') {
  return ctx.reply(
    'This order is no longer waiting for payment.'
  )
}
pendingReceiptOrders[ctx.from.id] = id
await ctx.reply(
  `■■■ Please send your payment receipt screenshot here.\n\n` +
  `■■■ Order ID: \`${id}\`\n\n` +
  `■■■ Your receipt will be checked automatically.\n` +
  `If it cannot be verified confidently, it will be sent to admin for manual verification.`
, {
  parse_mode: 'Markdown'
})
})

async function processReceiptMedia(ctx, media) {
  const id = pendingReceiptOrders[ctx.from.id]
  if (!id) return false
  if (
    media.type === 'document' &&
    !media.mimeType?.startsWith('image/')
  ) {
    await ctx.reply(
      '■■■ Please send the receipt as a photo or image file.'
    )
    return true
  }
  const db = loadDB()
  const order = db.orders.find(
    o => o.id === id && o.userId === ctx.from.id
  )
  if (!order) {
    delete pendingReceiptOrders[ctx.from.id]
    await ctx.reply('■■■ Order not found.')
    return true
  }
  if (order.status !== 'waiting_payment') {
    delete pendingReceiptOrders[ctx.from.id]
    await ctx.reply(
      '■■■ This order is no longer waiting for payment.'
    )
    return true
  }
  /* * IMPORTANT: * We do NOT hard-reject an uploaded receipt just because * the
  order's 10-minute payment window has passed. * * A receipt outside the allowed time
  goes to the admin * for manual verification instead. */
  const orderWindowExpired =
    Boolean(order.paymentExpiresAt) &&
    Date.now() > Number(order.paymentExpiresAt)
  let receiptData = null
  let aiError = null

```

```

try {
const fileLink = await bot.telegram.getFileLink(
media.fileId
)
const receiptCheck =
await checkPaymentReceipt(fileLink.href)
const cleaned = String(receiptCheck || '')
.replace(/^```json\s*/i, '')
.replace(/^```\s*/i, '')
.replace(/^```s*$/i, '')
.trim()
receiptData = JSON.parse(cleaned)
} catch (error) {
console.error(
'Receipt AI verification error:',
error
)
aiError = error
receiptData = null
}

let ref = ''
let sameAmount = false
let usedReference = null
let receiptTimeValid = false
let recipientNumberValid = true
let isReceipt = false
let confidence = 0
let confidenceOkay = false

if (receiptData) {
/* * The AI response uses reference_number. * "reference" is kept only as backward
compatibility. */
ref = normalizeReference(
receiptData.reference_number ||
receiptData.reference
)

usedReference = ref
? db.orders.find(o =>
o.id !== order.id &&
normalizeReference(
o.receipt?.reference
) === ref
)
: null

receiptData.duplicateReference =
Boolean(usedReference)

const extractedAmount =
parseReceiptAmount(receiptData.amount)
sameAmount =
Number.isFinite(extractedAmount) &&
extractedAmount === Number(order.totalPrice)

if (receiptData.datetime) {
const receiptDate = new Date(receiptData.datetime)
if (Number.isFinite(receiptDate.getTime())) {
const nowPH = new Date(
new Date().toLocaleString("en-US", {
timeZone: "Asia/Manila"
}))
}
)

const receiptPH = new Date(
receiptDate.toLocaleString("en-US", {
timeZone: "Asia/Manila"

```



```

    })
  )
  const ageMs = nowPH - receiptPH
  receiptTimeValid =
    ageMs >= 0 &&
    ageMs <= 10 * 60 * 1000
  }
}

isReceipt =
  receiptData.is_receipt === true
confidence =
  Number(receiptData.confidence)
/* * Accept 0-1 or 0-100 just in case the AI * unexpectedly returns a percentage. */
if (
  Number.isFinite(confidence) &&
  confidence > 1
) {
  confidence = confidence / 100
}

confidenceOkay =
  Number.isFinite(confidence) &&
  confidence >= 0.5
/* * Recipient number is checked only when the * receipt actually exposes a readable
number. * Missing/masked number alone does not reject it. */
const extractedNumber =
  normalizePhone(
    receiptData.recipient_number
  )
const expectedNumber =
  normalizePhone(GCASH_NUMBER)
if (
  extractedNumber &&
  expectedNumber &&
  expectedNumber !== '09'
) {
  recipientNumberValid =
    extractedNumber.slice(-10) ===
    expectedNumber.slice(-10)
}
}

const automaticVerificationPassed =
  Boolean(receiptData) &&
  isReceipt &&
  confidenceOkay &&
  sameAmount &&
  receiptTimeValid &&
  Boolean(ref) &&
  !receiptData.duplicateReference &&
  recipientNumberValid &&
  !orderWindowExpired
/* * Always save the uploaded receipt/reference, * including receipts that need
admin review. * This allows duplicate-reference detection later. */
order.receipt = {
  fileId: media.fileId,
  fileUniqueId: media.fileUniqueId,
  mediaType: media.type,
  reference: ref,
  paymentMethod:
    receiptData?.payment_method || '',
  recipient:
    receiptData?.recipient || '',

```

```

recipientNumber:
receiptData?.recipient_number || '',
amount:
receiptData?.amount ?? '',
datetime:
receiptData?.datetime || '',
transactionStatus:
receiptData?.status || '',
confidence:
Number.isFinite(confidence)
? confidence
: 0,
receivedAt: new Date().toISOString()
}
/* * PASS = automatic payment approval. */
if (automaticVerificationPassed) {
order.status = 'paid'
order.receiptStatus = 'verified'
order.paymentVerifiedBy = 'ai'
order.paymentVerifiedAt =
new Date().toISOString()
saveDB(db)
delete pendingReceiptOrders[ctx.from.id]
await ctx.reply(
'■ Payment verified automatically! Your order is now processing.'
)
await confirmPayment(order)
return true
}
/* * FAIL / UNCERTAIN = manual verification. * Buyer is NOT automatically rejected.
*/
order.receiptStatus = 'pending_verification'
saveDB(db)
delete pendingReceiptOrders[ctx.from.id]
await ctx.reply(
'■ Receipt received. Waiting for admin verification.'
)
if (ADMIN_ID) {
const reasons = []
if (aiError || !receiptData) {
reasons.push(
'■■ AI could not read the receipt'
)
} else {
if (!isReceipt) {
reasons.push(
'■■ AI is not confident this is a payment receipt'
)
}
}
if (!confidenceOkay) {
reasons.push(
`■■ Low AI confidence (${Math.round(confidence * 100) || 0}%)`
)
}
if (!sameAmount) {
reasons.push(
'■■ Amount needs checking'
)
}
if (!receiptTimeValid) {
reasons.push(

```

```

'■■ Receipt time is unreadable or outside 10 minutes'
)
}
if (orderWindowExpired) {
  reasons.push(
    '■■ Order payment window has passed'
  )
}
if (
  receiptData.duplicateReference
) {
  reasons.push(
    '■■ Reference number was already used'
  )
}
if (!ref) {
  reasons.push(
    '■■ Reference number could not be read'
  )
}
if (!recipientNumberValid) {
  reasons.push(
    '■■ Recipient number needs checking'
  )
}
}
if (!reasons.length) {
  reasons.push(
    '■■ Automatic verification was inconclusive'
  )
}
const adminCaption =
`■ FOR MANUAL VERIFICATION\n\n` +
`Order: ${id}\n` +
`Product: ${order.productName}\n` +
`Expected total: ■${order.totalPrice}\n\n` +
`AI payment method: ${receiptData?.payment_method || 'Unreadable'}\n` +
`AI amount: ${receiptData?.amount || 'Unreadable'}\n` +
`AI date/time: ${receiptData?.datetime || 'Unreadable'} (Philippine Time)\n` +
`AI reference: ${ref || 'Unreadable'}\n` +
`AI confidence: ${Math.round(confidence * 100) || 0}%\n\n` +
`Reason:\n${reasons.join('\n')}\n\n` +
`Buyer: ${order.firstName || ''} ${order.lastName || ''} ` +
`@${order.username || 'no_username'} (${order.userId})`
/* * Manual fallback has Approve. * There is NO automatic rejection. */
const keyboard = Markup.inlineKeyboard([
[
Markup.button.callback(
  '■ Approve Payment',
  `approve_${id}`
)
]
])
const sendReceipt =
media.type === 'document'
? bot.telegram.sendDocument(
  ADMIN_ID,
  media.fileId,
  {
    caption: adminCaption,
    ...keyboard
  }
)

```

```

    )
    : bot.telegram.sendPhoto(
      ADMIN_ID,
      media.fileId,
      {
        caption: adminCaption,
        ...keyboard
      }
    )
    await sendReceipt.catch(error => {
      console.error(
        'Failed to send receipt to admin:',
        error
      )
    })
  }
  return true
}

bot.action(/^approve_(.+)$/, async (ctx) => {
  if (!isAdmin(ctx)) {
    return ctx.answerCbQuery('Unauthorized')
  }
  const id = ctx.match[1]
  const db = loadDB()
  const order = db.orders.find(
    o => o.id === id
  )
  if (!order) {
    return ctx.answerCbQuery(
      'Order not found'
    )
  }
  /* * Prevent a second click from delivering * stock twice. */
  if (
    [
      'paid',
      'preparing',
      'waiting_gmail',
      'delivered'
    ].includes(order.status)
  ) {
    return ctx.answerCbQuery(
      'Order already processed'
    )
  }
  order.status = 'paid'
  order.receiptStatus = 'verified'
  order.paymentVerifiedBy = 'admin'
  order.paymentVerifiedAt =
    new Date().toISOString()
  saveDB(db)
  await ctx.answerCbQuery(
    'Payment approved'
  )
  await ctx
    .editMessageReplyMarkup({
      inline_keyboard: []
    })
    .catch(() => {})
  await ctx.reply(
    `■ Order ${id} approved. Processing delivery...`
  )

```

```

    )
    await confirmPayment(order)
  })
  bot.on('photo', async (ctx, next) => {
    const photo = ctx.message.photo[ctx.message.photo.length - 1]
    const handled = await processReceiptMedia(ctx, {
      type: 'photo',
      fileId: photo.file_id,
      fileUniqueId: photo.file_unique_id
    })
    if (!handled) return next()
  })
  bot.on('document', async (ctx, next) => {
    const document = ctx.message.document
    const handled = await processReceiptMedia(ctx, {
      type: 'document',
      fileId: document.file_id,
      fileUniqueId: document.file_unique_id,
      mimeType: document.mime_type || ''
    })
    if (!handled) return next()
  })
  // =====
  // MY ORDERS
  // =====
  bot.action('my_orders', async (ctx) => {
    await ctx.answerCbQuery()
    const db = loadDB()
    const orders = db.orders
      .filter(o => o.userId === ctx.from.id)
      .slice(-10)
      .reverse()
    if (!orders.length) {
      return ctx.reply('■ You have no orders yet.')
    }
    const statusMap = {
      waiting_payment: '■ Waiting for Payment',
      paid: '■ Payment Confirmed',
      preparing: '■ Preparing Delivery',
      waiting_gmail: '■ Waiting Gmail',
      delivered: '■ Delivered',
      cancelled: '■ Cancelled'
    }
    const lines = orders.map(o => {
      const qty = o.quantity || 1
      const price = o.pricePerItem || 0
      const total = o.totalPrice || price * qty
      const receipt =
        o.receiptStatus === 'pending_verification'
        ? '\n■ Receipt Pending Admin Review'
        : ''
      return (
        `■ \`${o.id}\`\n` +
        `${o.productName}\n` +
        `${qty} x ■${price} = ■${total}\n` +
        `${statusMap[o.status] || o.status}` +
        receipt
      )
    })
    await ctx.reply(

```

```

`■ *MY ORDERS*\n\n${lines.join('\n\n')}``,
{
  parse_mode: 'Markdown'
}
)
})

// =====
// QUANTITY ORDER HANDLER
// =====
bot.on('text', async (ctx, next) => {
  // Ignore commands like /admin /addstock /menu
  if (ctx.message.text?.startsWith('/')) {
    return next()
  }
  const pending = pendingOrders[ctx.from.id]
  if (!pending) return next()
  const text = (ctx.message.text || '').trim()
  const q = Number(text)
  if (
    !Number.isInteger(q) ||
    q < 1 ||
    q > 50
  ) {
    return ctx.reply(
      '■ Invalid quantity. Enter 1-50.'
    )
  }

  const product =
    PRODUCTS[pending.productId]
  if (!product) {
    delete pendingOrders[ctx.from.id]
    return ctx.reply(
      'Product not found.'
    )
  }

  const db = loadDB()
  // Check stock for instant delivery items
  if (
    product.deliveryType === 'link' ||
    product.deliveryType === 'email_password'
  ) {
    const available =
      db.stock.filter(
        s =>
          s.productId === product.id
      ).length
    if (q > available) {
      return ctx.reply(
        `■ Not enough stock.\n\nAvailable: ${available}\nRequested: ${q}`
      )
    }
  }

  const order = {
    id: orderId(),
    userId: ctx.from.id,
    username:
      ctx.from.username || '',
    firstName:
      ctx.from.first_name || '',
  }

```

```

lastName:
ctx.from.last_name || '',
productId:
product.id,
productName:
product.name,
quantity: q,
pricePerItem:
product.price,
totalPrice:
product.price * q,
status:
'waiting_payment',
createdAt:
new Date().toISOString(),
paymentExpiresAt:
Date.now() +
(10 * 60 * 1000),
gmail: null,
deliveredItems: []
}
db.orders.push(order)
if (!db.users) {
db.users = []
}
const exists =
db.users.find(
u => u.id === ctx.from.id
)
if (!exists) {
db.users.push({
id: ctx.from.id,
username:
ctx.from.username || '',
firstName:
ctx.from.first_name || '',
lastName:
ctx.from.last_name || '',
createdAt:
new Date().toISOString()
})
}
saveDB(db)
delete pendingOrders[ctx.from.id]
await ctx.reply(
`🍱 *ORDER CREATED!*👤👤` +
`${product.emoji} Product: *${product.name}*👤` +
`🍱 Quantity: *${q}*👤` +
`🍱 Price: *🍱${product.price} each*👤` +
`🍱 Total: *🍱${order.totalPrice}*👤👤` +
`🍱 Order ID: \`${order.id}\`👤👤` +
`🍱 Waiting for Payment`,
{
parse_mode: 'Markdown',
...Markup.inlineKeyboard([

```

```

[
Markup.button.callback(
'■■ Payment Guide',
'payment_guide'
)
],
[
Markup.button.callback(
'■■ Cancel Order',
`cancel:${order.id}`
)
]
])
}
)

if (ADMIN_ID) {
await bot.telegram.sendMessage(
ADMIN_ID,
`■■ New Order\n\n` +
`Order: ${order.id}\n` +
`Product: ${product.name}\n` +
`Qty: ${q}\n` +
`Total: ■■${order.totalPrice}\n` +
`Buyer: @${order.username || 'no_username'} (${order.userId})`
).catch(() => {})
}
})

// =====
// GMAIL HANDLER (CANVA)
// =====

bot.hears(
/^GMAIL\s+(HYU-[A-Z0-9]+)\s+([\^s@]+\@[^\s@]+\.[^\s@]+)$/i,
async (ctx) => {
const [, id, gmail] =
ctx.match

const db = loadDB()

const order =
db.orders.find(
o =>
o.id === id &&
o.userId === ctx.from.id
)

if (!order) {
return ctx.reply(
'Order not found.'
)
}

if (order.productId !== 'canva') {
return ctx.reply(
'This order does not require Gmail.'
)
}

if (
order.status !== 'waiting_gmail'
) {
return ctx.reply(

```



```

    'This order is not waiting for Gmail.'
  )
}

order.gmail = gmail
order.status =
'preparing'
saveDB(db)

await ctx.reply(
`■■ Gmail received!\n\n` +
`Order: \`${id}\`\n` +
`Gmail: \`${gmail}\`\n\n` +
`■■ Preparing your Canva invite.`,
{
  parse_mode: 'Markdown'
}
)

if (ADMIN_ID) {
  await bot.telegram.sendMessage(
    ADMIN_ID,
    `■■ Canva Gmail Received\n\n` +
    `Order: ${id}\n` +
    `Gmail: ${gmail}`
  ).catch(() => {})
}
}
)

// =====
// RECEIVED BUTTON
// =====

bot.action(
/^received:(HYU-.+)$/,
async (ctx) => {
  await ctx.answerCbQuery(
    'Thank you! ■■'
  )

  await ctx.reply(
    '■■ Thank you for confirming. Enjoy your order! ■■'
  )
}
)

// =====
// ADMIN STOCK COMMANDS
// =====

bot.command('admin', async (ctx) => {
  if (!isAdmin(ctx)) {
    return ctx.reply('■■ Unauthorized')
  }

  await ctx.reply(
    `■■ ADMIN PANEL\n\n` +
    `/stock - View stock\n` +
    `/addlink - Add link stock\n` +
    `/addaccount - Add email/password stock`
  )
})

bot.command('stock', async (ctx) => {
  if (!isAdmin(ctx)) {
    return
  }

```

```

}
const db = loadDB()
const counts = {}
for (const item of db.stock) {
  counts[item.productId] =
    (counts[item.productId] || 0) + 1
}
let text =
  '■■ STOCK LIST\n\n'
for (const p of Object.values(PRODUCTS)) {
  text +=
    `${p.emoji} ${p.name}: ${counts[p.id] || 0}\n`
}
await ctx.reply(text)
})

// Add link stock
bot.command('addlink', async (ctx) => {
  if (!isAdmin(ctx)) {
    return ctx.reply('■■ Unauthorized')
  }
  delete pendingOrders[ctx.from.id]
  adminStockFlow[ctx.from.id] = {
    type: 'gemini_links'
  }
  await ctx.reply(
    '■■ Send Gemini links now.\n\n' +
    'One link per line.\n' +
    'You can add up to 50 links at once.'
  )
})

// Add email/password stock
bot.command('addaccount', async (ctx) => {
  if (!isAdmin(ctx)) {
    return ctx.reply('■■ Unauthorized')
  }
  adminStockFlow[ctx.from.id] = {
    type: 'email_password'
  }
  await ctx.reply(
    'Send product id email password:\n\nExample:\ncapcut test@gmail.com pass123'
  )
})

// Admin stock input
bot.on('text', async (ctx, next) => {
  const flow = adminStockFlow[ctx.from.id]
  if (!flow) {
    return next()
  }
  if (!isAdmin(ctx)) {
    return next()
  }
  // GEMINI LINK STOCK
  if (flow.type === 'gemini_links') {
    const links = ctx.message.text
      .split('\n')
      .map(x => x.trim())
      .filter(x =>
        x.startsWith('http://') ||

```

```

x.startsWith('https://')
)
.slice(0,50)
if (!links.length) {
return ctx.reply(
'■■ Send links only.'
)
}

const db = loadDB()
for (const link of links) {
db.stock.push({
productId: 'gemini',
type: 'link',
link,
addedAt: new Date().toISOString()
})
}

saveDB(db)
await notifyAllUsers(
`■■ HYUNA STORE UPDATE ■■\n\n` +
`■■ New stock available!\n\n` +
`■■ ORDER NOW\n\n` +
`https://t.me/AITOOLSHyuna_Bot?start=shop\n\n` +
`■■ Thank you for supporting us!`
);

delete adminStockFlow[ctx.from.id]

return ctx.reply(
`■■ Added ${links.length} Gemini links and notified users.`
)
}

return next()
})

// =====
// DELIVERY FUNCTION
// =====

async function confirmPayment(order) {
const db = loadDB()
const fresh =
db.orders.find(
o => o.id === order.id
)

if (!fresh) {
return
}

const product =
PRODUCTS[fresh.productId]

if (!product) {
return
}

// Manual products
if (
product.deliveryType === 'manual'
) {
fresh.status =
'preparing'
saveDB(db)
await bot.telegram.sendMessage(

```

```

fresh.userId,
`■■ Payment confirmed!\n\n` +
`Order: ${fresh.id}\n\n` +
`■■ Your order is being prepared manually.`
)
return
}

// Canva
if (
product.deliveryType === 'manual_invite'
) {
fresh.status =
'waiting_gmail'
saveDB(db)
await bot.telegram.sendMessage(
fresh.userId,
`■■ Payment confirmed!\n\n` +
`Order: ${fresh.id}\n\n` +
`Please send your Gmail:\n\n` +
`GMAIL ${fresh.id} your@gmail.com`
)
return
}

// Stock delivery
const available =
db.stock.filter(
s =>
s.productId === fresh.productId
)

const items =
available.slice(
0,
fresh.quantity
)

if (
items.length < fresh.quantity
) {
fresh.status =
'preparing'
saveDB(db)
await bot.telegram.sendMessage(
fresh.userId,
`■■ Payment confirmed.\n\n` +
`■■ Preparing delivery.`
)
return
}

fresh.deliveredItems =
items

db.stock =
db.stock.filter(
s => !items.includes(s)
)

fresh.status =
'delivered'

```

```

fresh.deliveredAt =
new Date().toISOString()
saveDB(db)

let delivery =
`■■ PAYMENT CONFIRMED!\n\n` +
`■■ Order: ${fresh.id}\n\n`

for (const item of items) {
  if (item.type === 'link') {
    delivery +=
`■■ Link:\n${item.link}\n\n`
  }
  if (
    item.type === 'email_password'
  ) {
    delivery +=
`■■ Email: ${item.email}\n` +
`■■ Password: ${item.password}\n\n`
  }
}

await bot.telegram.sendMessage(
  fresh.userId,
  delivery
)
}

// =====
// START BOT
// =====

bot.catch((err) => {
  console.error(
    'BOT ERROR:',
    err
  )
})

bot.launch()
console.log(
  '■■ Hyuna Store Bot Started'
)

process.once(
  'SIGINT',
  () => bot.stop('SIGINT')
)

process.once(
  'SIGTERM',
  () => bot.stop('SIGTERM')
)

```